Session 3: How the Web Talks (HTTP & REST)

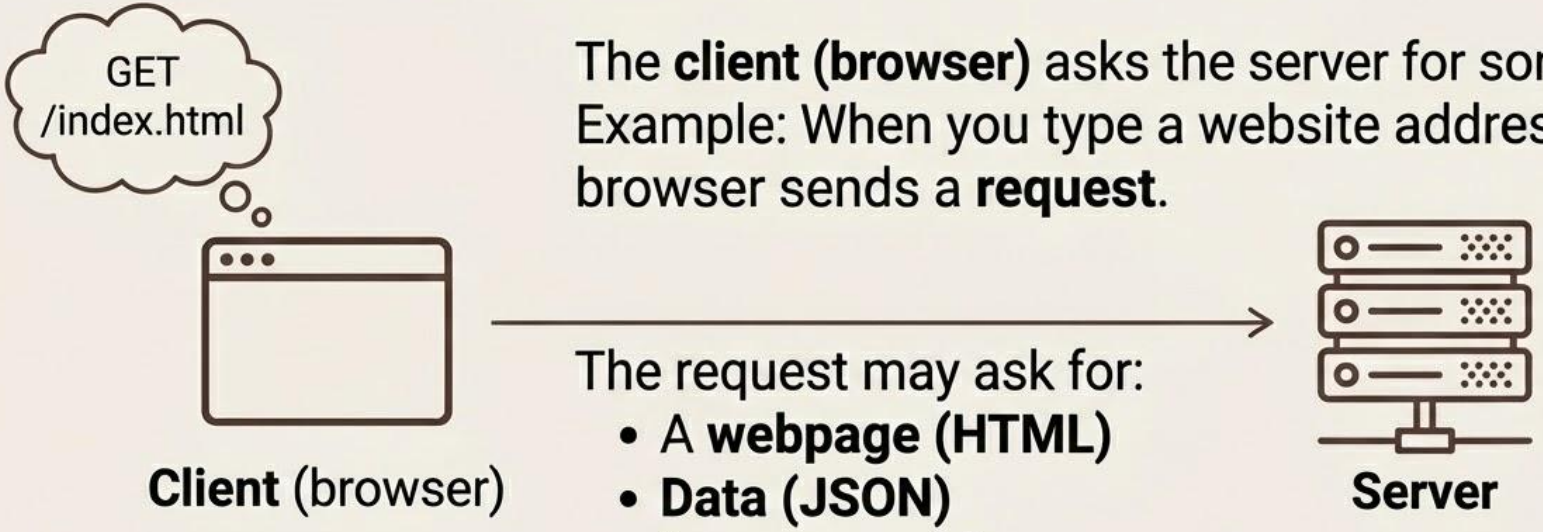
1. The Request–Response Cycle

The internet works like **sending letters** and **receiving replies**. 📫

Client – The user side (browser or mobile app).

Server – The computer that stores and sends the website data.

Step 1: Client Sends a Request



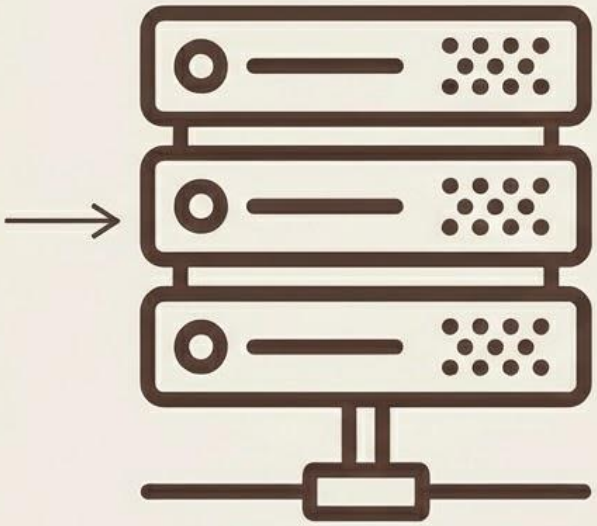
The **client (browser)** asks the server for something.
Example: When you type a website address, the browser sends a **request**.

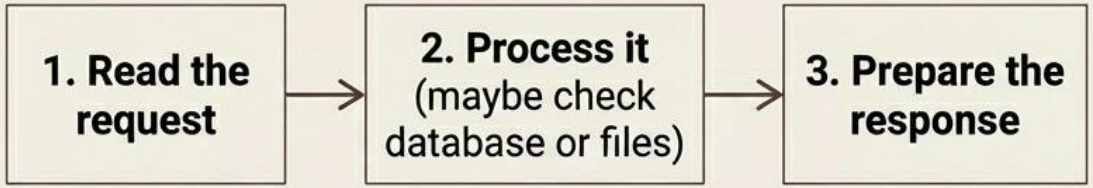
The request may ask for:

- A **webpage (HTML)**
- **Data (JSON)**
- **Images** or **videos**

Example request: GET /index.html
This means: 👉 "Server, please send me the homepage."

Step 2: Server Processes the Request

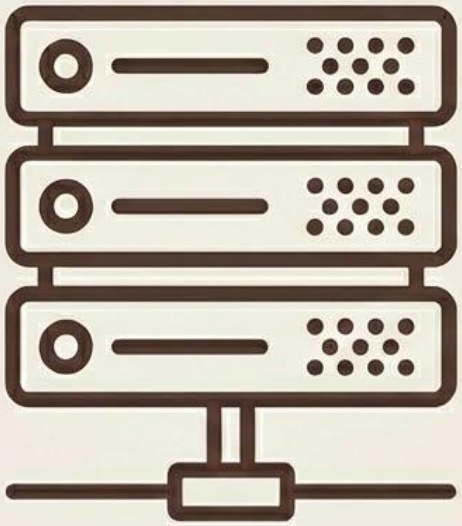




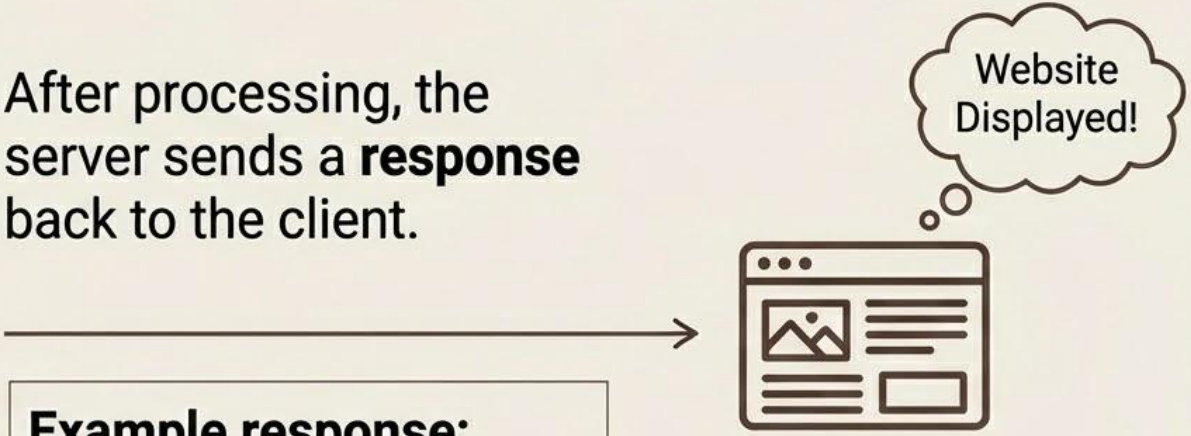
The **server (for example Node.js)** receives the request. The server will:

1. **Read the request**
2. **Process it** (maybe check database or files)
3. **Prepare the response**

Step 3: Server Sends a Response



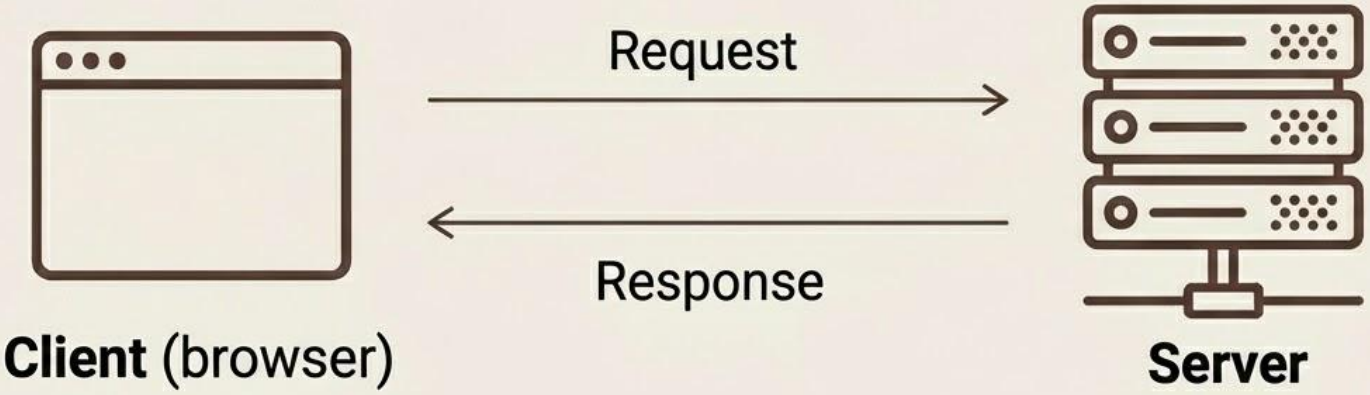
After processing, the server sends a **response** back to the client.



Example response:
200 OK
Content-Type: text/html

Then it sends the webpage data.
The browser receives it and **displays the website**.

Simple Flow



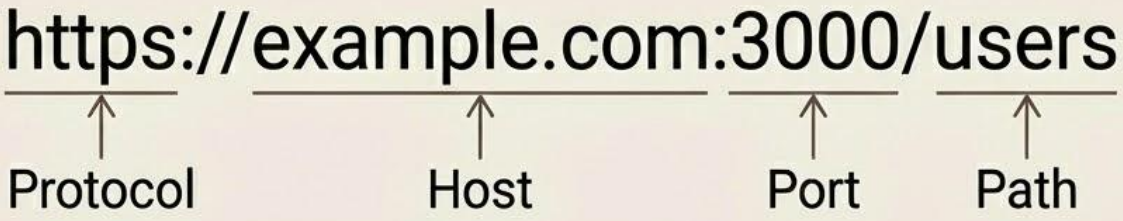
Example: google.com
1. You type: google.com (Browser → Request webpage)
2. Server → Sends webpage
3. Browser → Shows the page

The 'Address' of the Web: URL

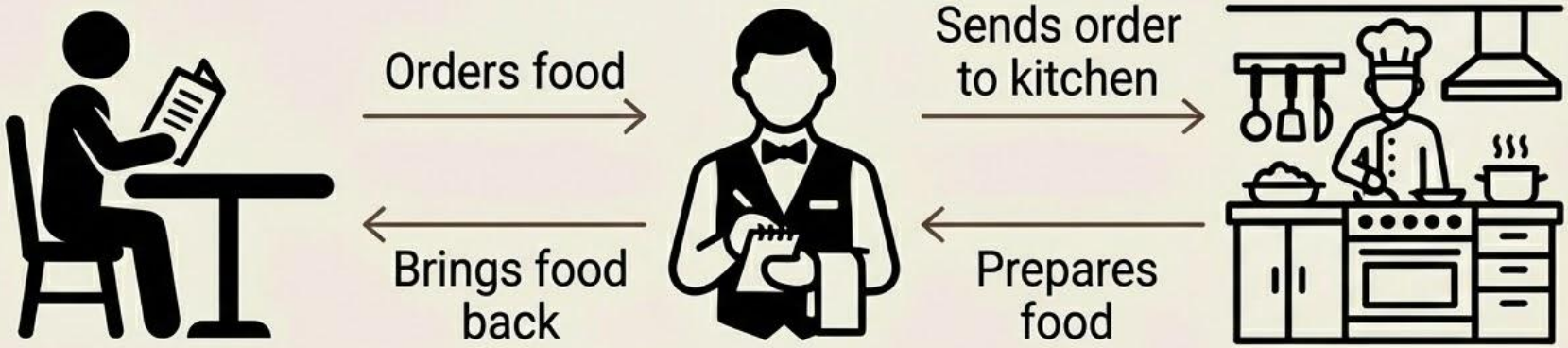
Every request needs an **address** called a **URL (Uniform Resource Locator)**.

https://example.com:3000/users

Part	Meaning	Example
Protocol	Communication method	`http` or `https`
Host	Server name	`example.com`
Port	Server port number	`3000`
Path	Specific resource	`/users`



Real-Life Example



Imagine a **restaurant** 🍔.

- **Customer (Client)** → Orders food.
- **Waiter** → Sends order to kitchen.
- **Kitchen (Server)** → Prepares food.
- **Waiter** → Brings food back.

Customer = Browser, Kitchen = Server

HTTP Methods (HTTP Verbs)

Define the type of action a client wants to perform on a server.

Also known as HTTP Verbs.

Understanding the ‘Payload’ in Computing

The actual intended message or data being sent, excluding the **‘overhead’** (headers, metadata, routing info).

Web Development (HTTP/API):
Body of a request (e.g., POST/PUT). Often JSON/XML.
Example JSON: { “username”: “dev_pro”, … }

Networking (OSI Model):
Relative to the layer. Ethernet Frame payload contains IP Packet, which contains TCP Segment, which contains Application Data.

Cybersecurity & Malware:
The malicious part of the code that performs the harmful action (e.g., encrypting files, stealing passwords).

Embedded Systems:
Specific data bytes in a transmission (e.g., sensor readings like 0x05 0x1B in a larger data stream).

Feature	Header / Overhead	Payload
Purpose	✓ Routing, sizing, security, protocol rules.	✓ The actual “meat” of the data.
Size	✓ Usually fixed or strictly limited.	✗ Can vary significantly.
Visibility	✓ Read by routers, servers, mid-points.	✗ Read by end application or recipient.

Common HTTP Methods: GET & POST

GET

Retrieves data from the server. Should not change anything on the server.

Example: Fetch a webpage or API data.

POST

Sends data to the server to create a resource. Often used for form submissions.

Example: Create a new user.

Common HTTP Methods: PUT, PATCH & DELETE

PUT

Updates an existing resource (or creates it if it doesn't exist). Replaces the entire resource.

Example: Update user profile completely.

PATCH

Partially updates a resource. Only modifies specific fields.

Example: Change just the email of a user.

DELETE

Removes a resource from the server.

Example: Delete a user account.

Other HTTP Methods

HEAD

Same as GET but returns only headers (no body). Useful for checking if a resource exists.

OPTIONS

Returns the supported HTTP methods for a resource. Used in CORS.

CONNECT

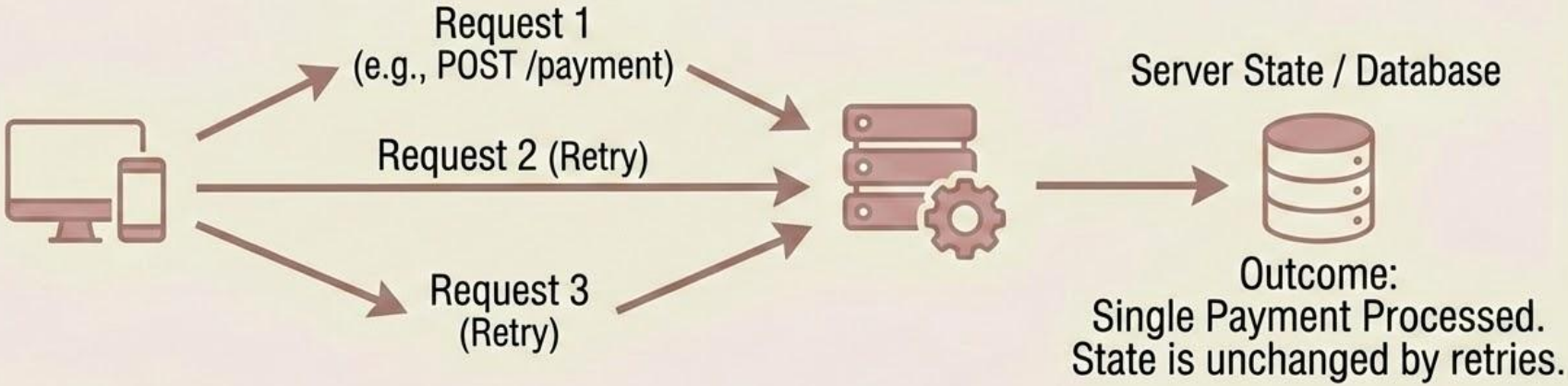
Establishes a tunnel (used for HTTPS through proxies).

TRACE

Echoes back the request for debugging.

Idempotency in Web APIs

The property where making multiple identical requests has the same intended effect on the server as making a single request.



Why it's Crucial:

- Building reliable and fault-tolerant systems.
- Essential in distributed environments.
- Handles network issues and automatic retries gracefully without unintended side effects.

HTTP Status Codes: Understanding Server Responses

A guide to the internet's communication outcomes

1xx: Informational

The request was received, and the process is **continuing.** These are rarely seen by end-users but are used by the underlying protocols.

- 101 Switching Protocols: Often seen when a connection is being upgraded to a WebSocket.

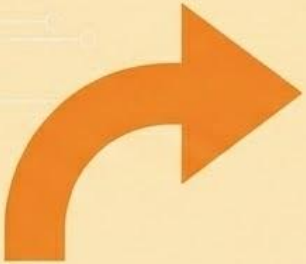
2xx: Success

The action was successfully received, understood, and **accepted**.

- **200 OK:** The standard response for a successful HTTP request.
- **201 Created**: The request was successful and a new resource was created (common for POST requests).
- **204 No Content**: The server successfully processed the request but isn't returning any content (common for DELETE or PUT updates).

3xx: Redirection

Further action needs to be taken to complete the request.



- **301 Moved Permanently:** This and all future requests should be directed to the given URI.
- **302 Found (Temporary Redirect):** The resource is temporarily under a different URI.
- **304 Not Modified:** Tells the browser that the cached version of the resource is still valid, saving bandwidth.

4xx: Client Error

The request contains bad syntax or cannot be fulfilled. This is usually the fault of the requester.

- **400 Bad Request:** The server cannot process the request due to a client error (e.g., malformed syntax).
- **401 Unauthorized:** Similar to 403, but specifically when authentication is required and has failed or hasn't been provided.
- **403 Forbidden:** The server understands the request but refuses to authorize it.
- **404 Not Found:** The most famous code; the server cannot find the requested resource.
- **429 Too Many Requests:** The user has sent too many requests in a given amount of time ('rate limiting').

5xx: Server Error

The server failed to fulfill an apparently valid request. This is a fault on the server's side.

- **500 Internal Server Error:** A generic error message when the server encounters an unexpected condition.
- **502 Bad Gateway:** The server, while acting as a gateway, received an invalid response from the upstream server.
- **503 Service Unavailable**: The server is currently unable to handle the request (due to maintenance or overloading).
- **504 Gateway Timeout**: The server was acting as a gateway and didn’t get a response in time.

Quick Summary Table

Class	Category	Meaning
1xx	Informational	Hold on...
2xx	Success	Here you go!
3xx	Redirection	Go over there.
4xx	Client Error	You messed up.
5xx	Server Error	I messed up.